

# SZÁMÍTÓGÉPES ARCHITEKTÚRÁK

---

A mikroarchitektúra szintje I.

Nagy Antal

# A mikroarchitektúra szintje

- Digitális logika feletti szint
- Feladata
  - A felette lévő ISA-szint (utasításrendszer-architektúra szintje) megvalósítása
- Függ
  - Az ISA szint megvalósításától
  - A számítógép ár- és teljesítmény-célkitűzéseitől
- Sok ISA olyan, egyszerű utasításokkal rendelkezik, amelyek egyetlen ciklusidő alatt végrehajthatók
  - RISC
- Összetettebb ISA-k egyetlen utasítás végrehajtásához több ciklust igényelhetnek
  - Pentium-4, CISC

# A mikroarchitektúra szintje

- Egy utasítás végrehajtása
  - Operandusok helyének meghatározása a memóriában
  - Operandusok memóriából való kiolvasása
  - Eredmény eltárolása a memóriába
- Utasításokon belüli műveletek sorba állítása
  - Vezérlés másfajta megközelítése

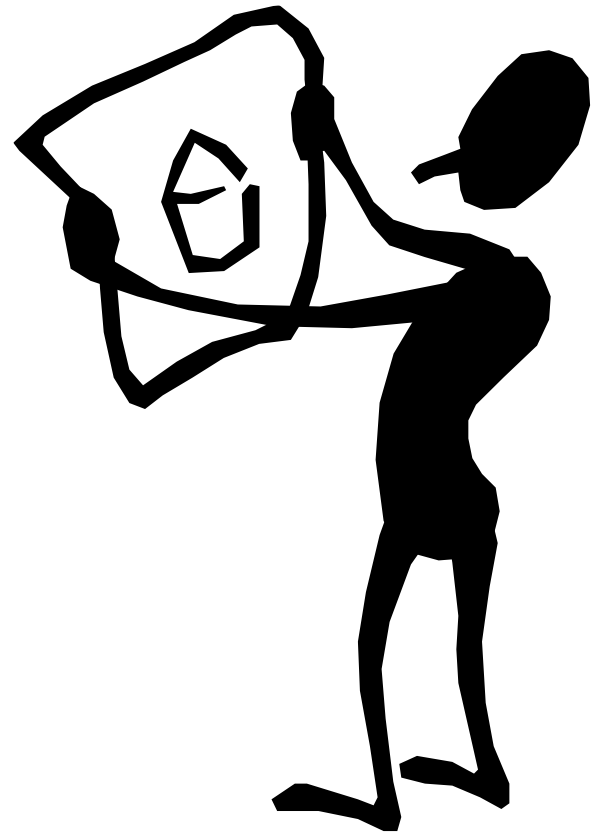


# Mikroarchitektúra példa

- Nincs általános tervezési alapelv
- ISA-példa
  - Egész utasításokat tartalmazó Java virtuális gép (IJVM)
- Mikroprogramot fog tartalmazni a mikroarchitektúránk
  - ROM-ban tároljuk
  - IJVM utasítások
    - Betöltése
    - Dekódolása
    - Végrehajtása
  - Kis mikroprogramra van szükség
    - Hatékonyan vezérli az egyes kapukat az aktuális hardverben
      - Csak az előző fejezetben leírt alapelemekből áll

# Mikroarchitektúra példa

- Tervezés, mint programozási probléma
  - ISA-szint minden egyes utasítása egy függvény, amit a főprogram hív meg
  - A főprogram egy végtelen ciklus
    - Meghatározza a meghívandó függvényt
    - Meghívja azt
    - Kezdi újra az egészet
- Mikroprogram
  - Változók egy halmaza
    - Számítógép állapota
  - Minden függvény el tudja érni
  - Megváltoztatja
    - Pl. utasítás számlálót (PC)
      - A következő végrehajtandó függvényre mutat



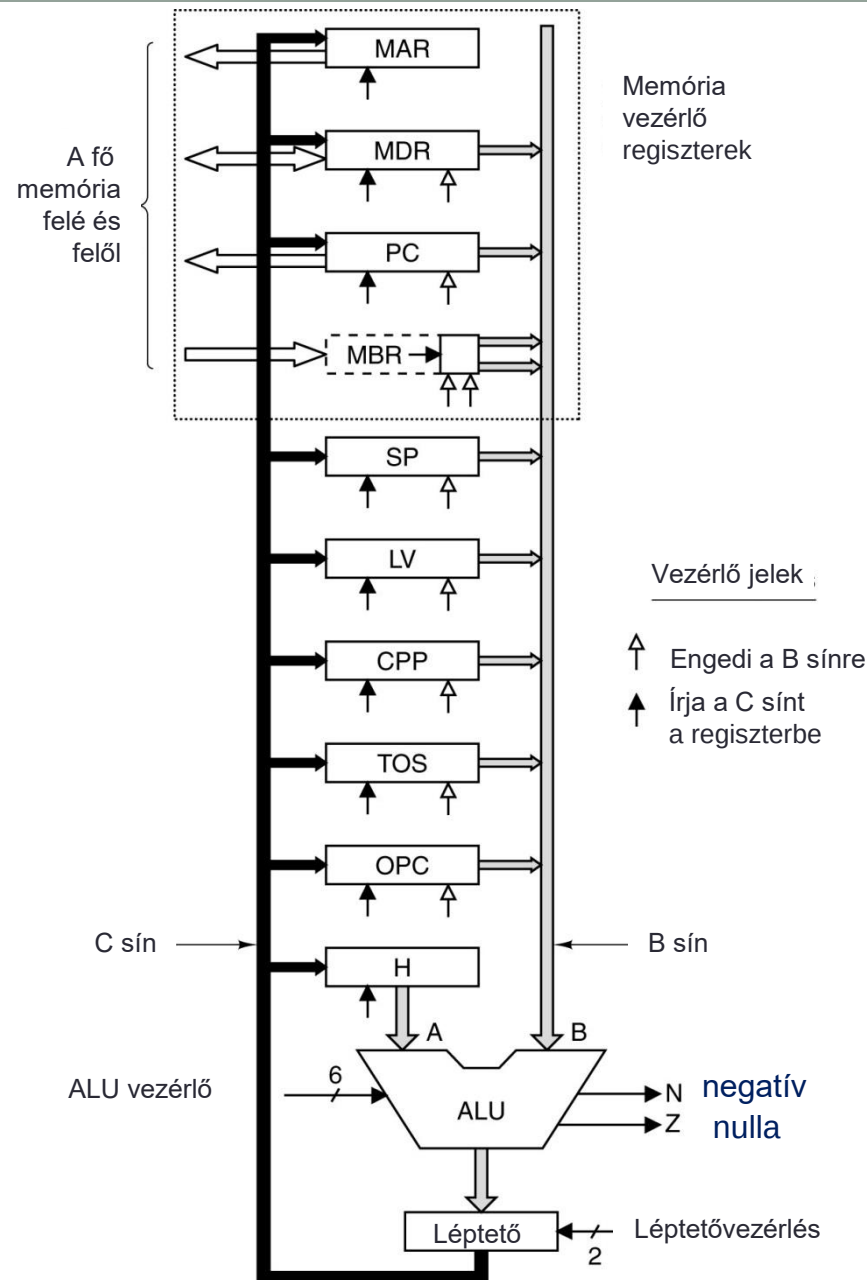
# Mikroarchitektúra példa

- Az IJVM utasításai rövidék
  - Egy vagy két mező
    - Speciális szereppel
  - Első mező
    - Műveleti kód
    - Azonosítja az utasítást
  - További mezők
    - Operandusok
      - Lokális változók
- Betölt-végrehajt ciklus
  - Alapja az ISA-k megvalósításának



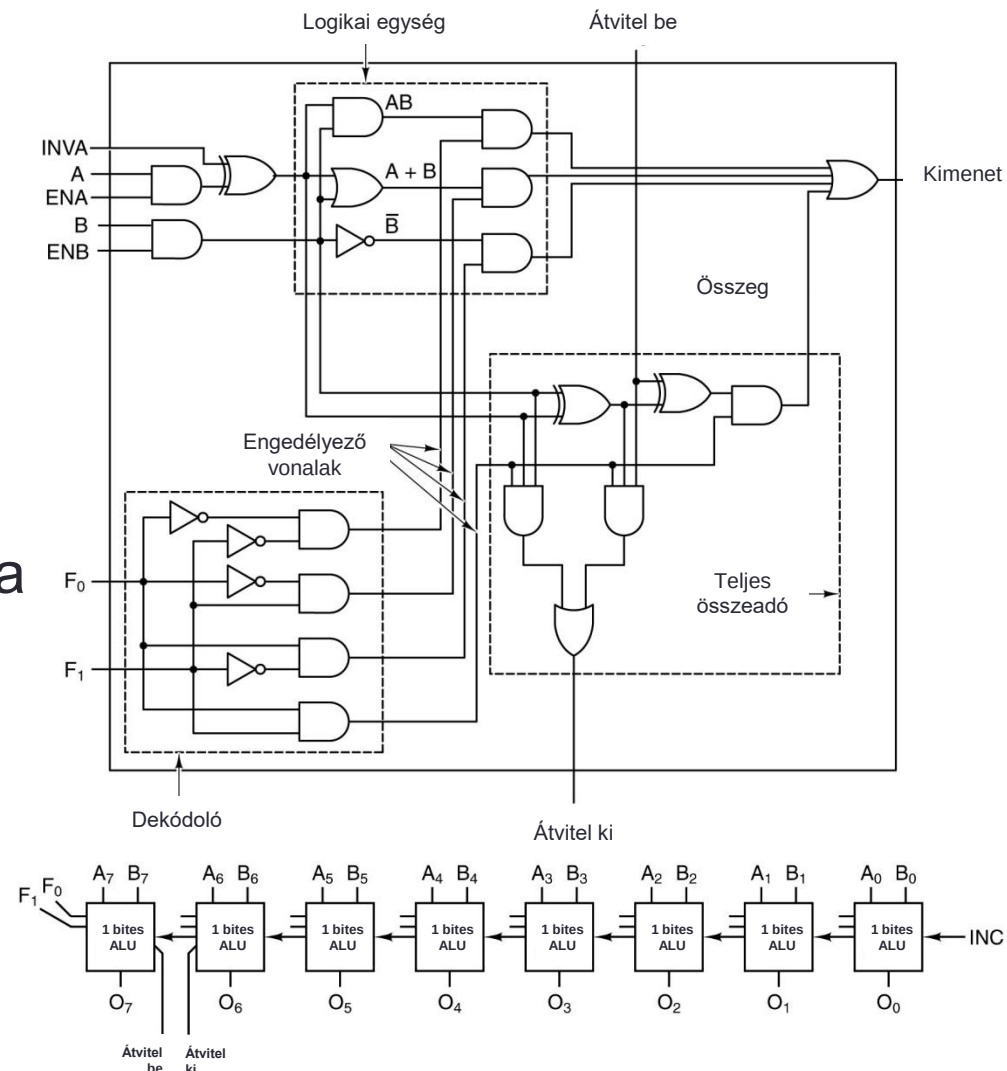
# Adatút/Data Path

- Tartalmazza az ALU-t
  - Bemeneteivel és kimeneteivel együtt
- Sok 32 bites regisztert tartalmaz
  - PC, SP, MDR
    - Ezek a regiszterek **csak** a mikroarchitektúra szintjén érhetők el a mikroprogrammal
  - Egyesek az ISA-szintű architektúra ugyanilyen nevű regiszterével egyezik meg
  - A regiszterek tartalmát a B sínre lehet irányítani
  - ALU kimenete a léptetőbe vezet
    - Ezután a C sínre vezet
      - Értéke egy időben több regiszterbe írható



# Adatút

- Az ALU 6-os címkéjű nyila a hat vezérlővonal
  - F0 és F1 meghatározza a műveletet
  - ENA és ENB engedélyezi a bemenetet
  - INVA invertálja az A bemenetet
  - INC a legalacsonyabb helyértékű biten az „átvitel be” bemenetet
- Az ALU-t vezérlő jelek nem mindegyike van kihasználva



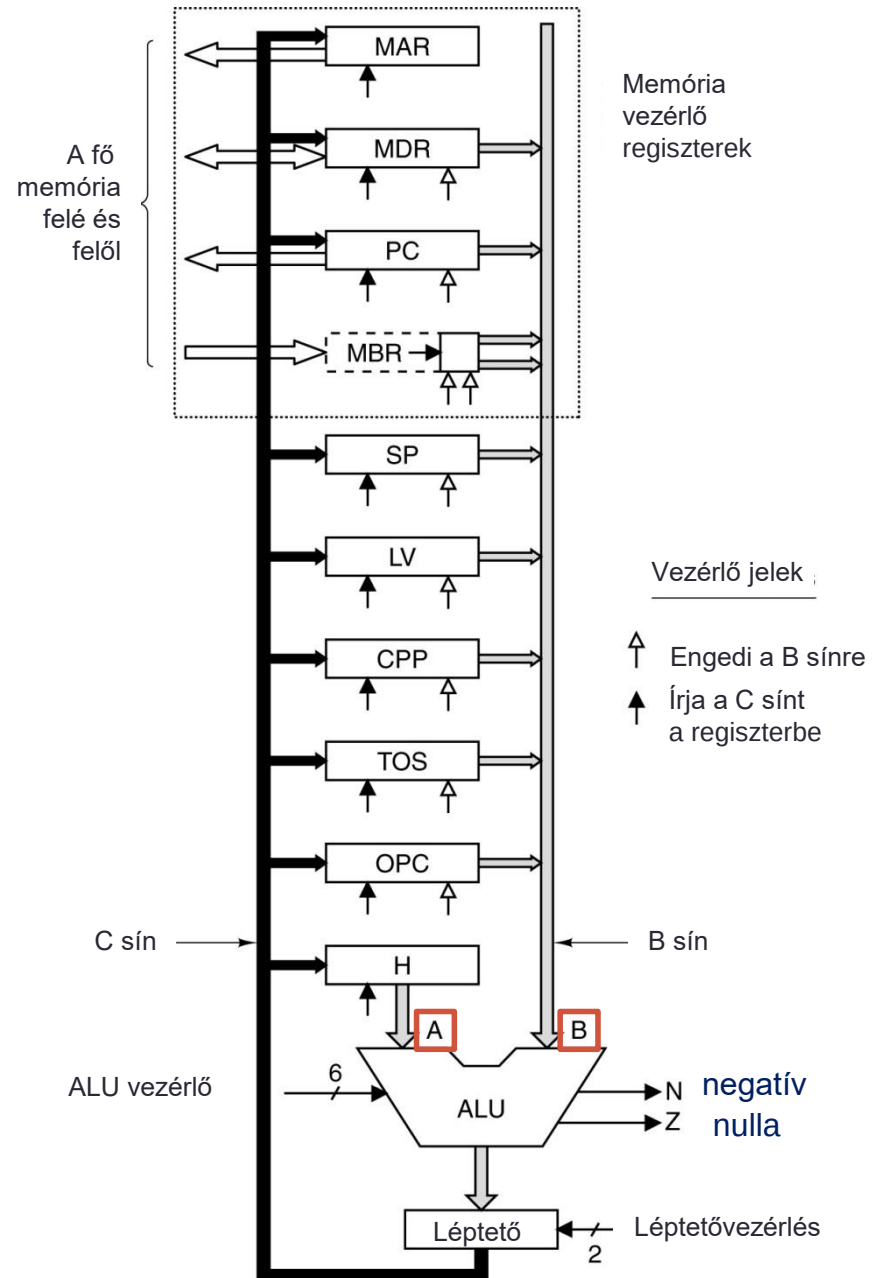
# Adatút

- Néhány érdekesebb kombináció
  - Több lehetőség van ugyanannak az eredménynek az eléréséhez
    - $-A$  az  $A$  kettes komplementjét jelenti

| $F_0$ | $F_1$ | ENA | ENB | INVA | INC | Function       |
|-------|-------|-----|-----|------|-----|----------------|
| 0     | 1     | 1   | 0   | 0    | 0   | A              |
| 0     | 1     | 0   | 1   | 0    | 0   | B              |
| 0     | 1     | 1   | 0   | 1    | 0   | $\overline{A}$ |
| 1     | 0     | 1   | 1   | 0    | 0   | $\overline{B}$ |
| 1     | 1     | 1   | 1   | 0    | 0   | A + B          |
| 1     | 1     | 1   | 1   | 0    | 1   | A + B + 1      |
| 1     | 1     | 1   | 0   | 0    | 1   | A + 1          |
| 1     | 1     | 0   | 1   | 0    | 1   | B + 1          |
| 1     | 1     | 1   | 1   | 1    | 1   | B - A          |
| 1     | 1     | 0   | 1   | 1    | 0   | B - 1          |
| 1     | 1     | 1   | 0   | 1    | 1   | $-A$           |
| 0     | 0     | 1   | 1   | 0    | 0   | A AND B        |
| 0     | 1     | 1   | 1   | 0    | 0   | A OR B         |
| 0     | 1     | 0   | 0   | 0    | 0   | 0              |
| 1     | 1     | 0   | 0   | 0    | 1   | 1              |
| 1     | 1     | 0   | 0   | 1    | 0   | -1             |

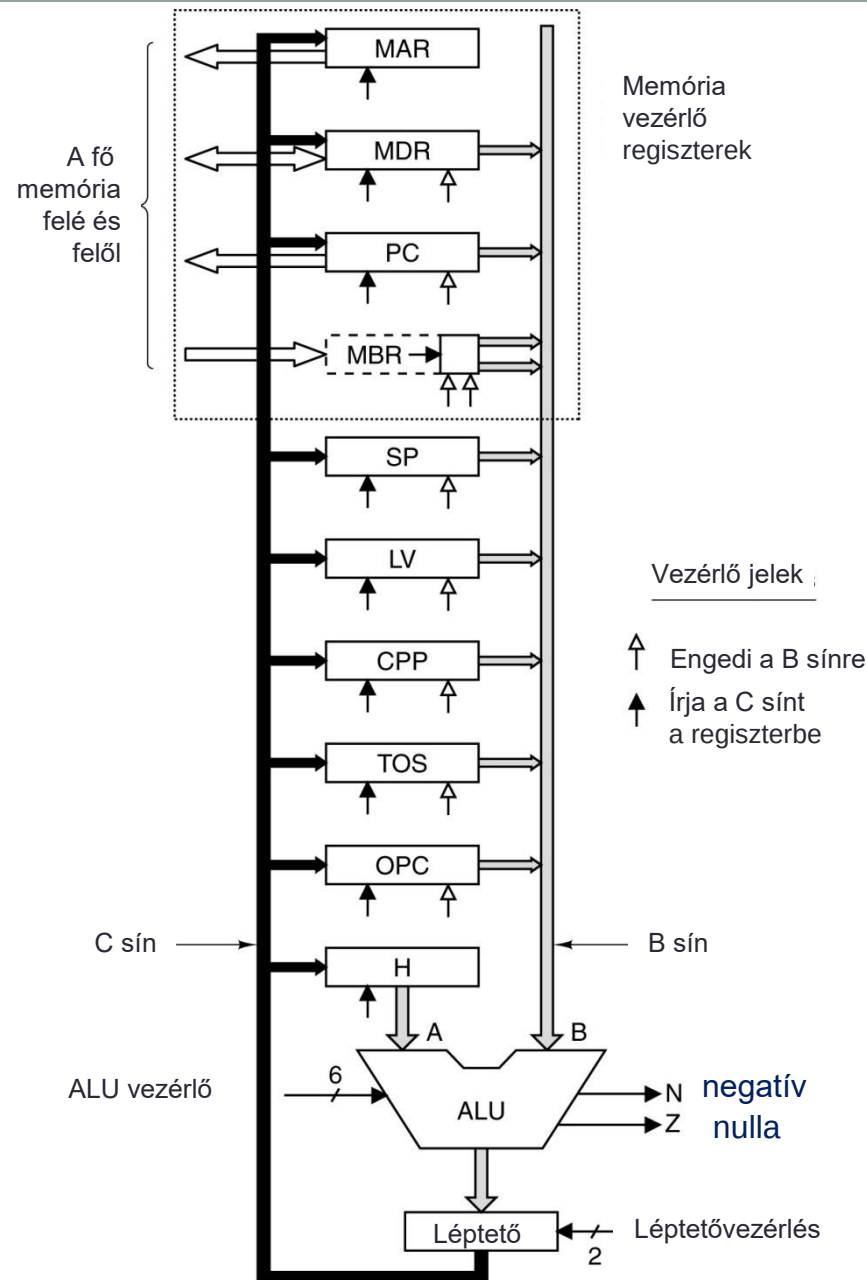
# Adatút

- Az ALU-nak két adatbemenetre van szüksége
  - A és B
  - A bal oldali bemenet (A) a H (Holding) tartó regiszterhez kapcsolódik
  - A jobb oldali bemenet a pedig a B sínhez
    - Képes betölteni a kilenc forrás bármelyikét



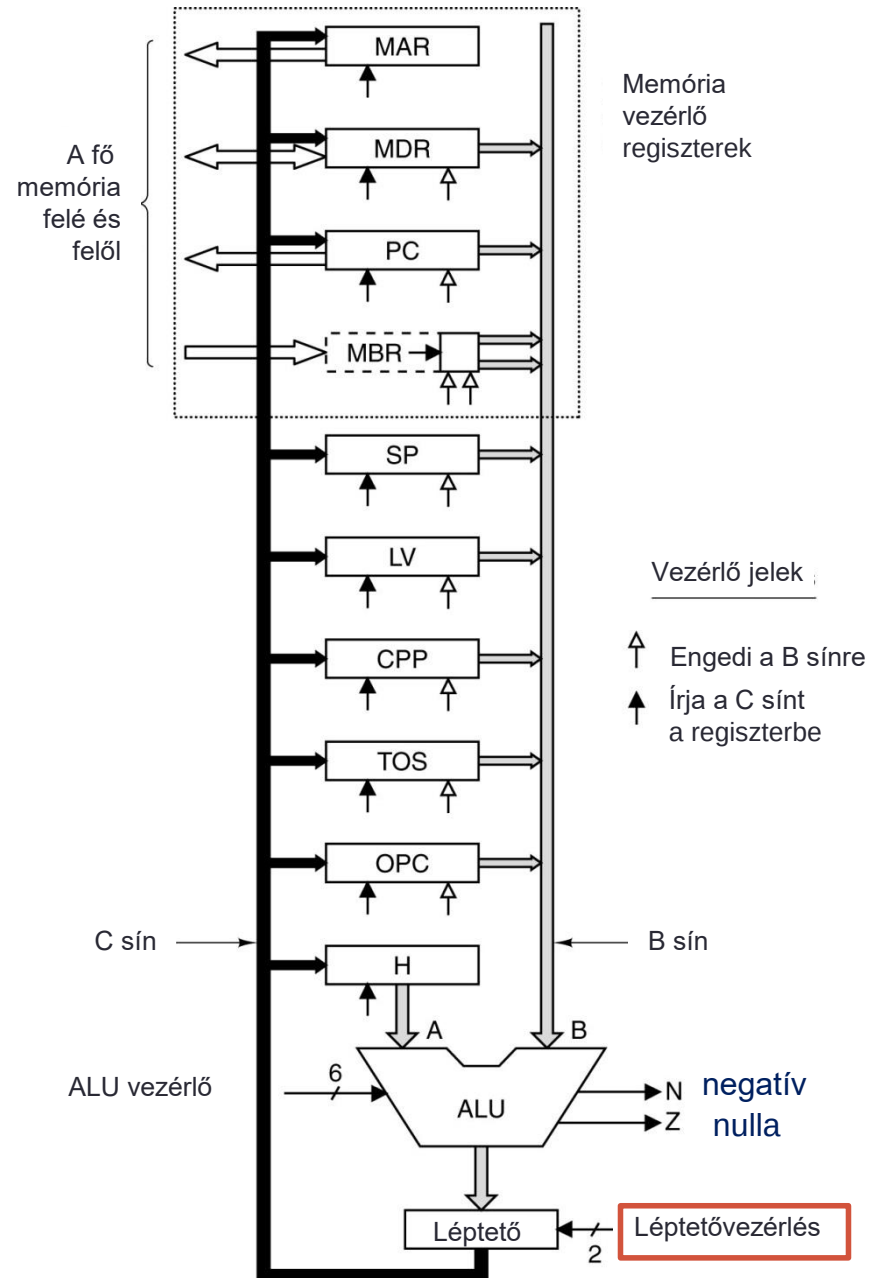
# Adatút

- H feltöltése
  - Választunk egy ALU tevékenységet
    - A jobb oldali bemenetet a B sínről csak átengedi
    - ALU bemenetek összeadása negált ENA-val
      - A baloldali bemenet 0 lesz
    - Nullát adva a B sín értékéhez
    - A léptetőn módosítás nélkül tovább küldve H-ban tároljuk



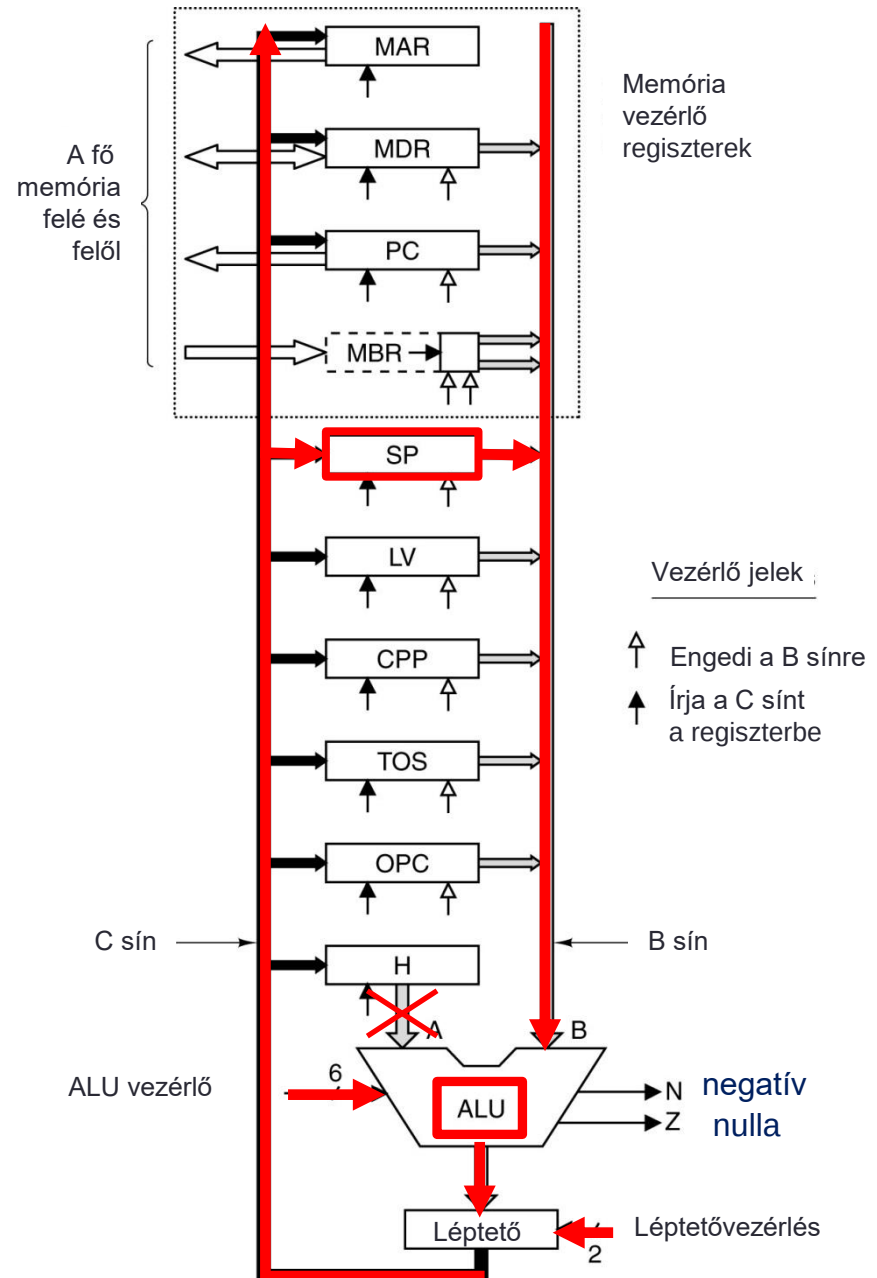
# Adatút

- ALU kimenetének irányítása (léptető)
  - SLL8
    - Logikai balra léptetés 8 bittel
    - Nullával tölti fel az alacsony helyiértékű biteket
  - SRA1
    - Aritmetikai jobbra léptetés 1 bittel
    - A legmagasabb helyiértékű bitet nem változtatja meg



# Adatút

- Ugyanazt a regisztert egy cikluson belül olvasni és írni is lehet
  - SP-t B sínre rakjuk
    - Kiválasztjuk SP-t
  - Tiltjuk az ALU bal oldali bemenetét
  - Engedélyezzük az INC-t
  - Az eredményt SP-be tároljuk
    - SP értékét eggyel növeltük



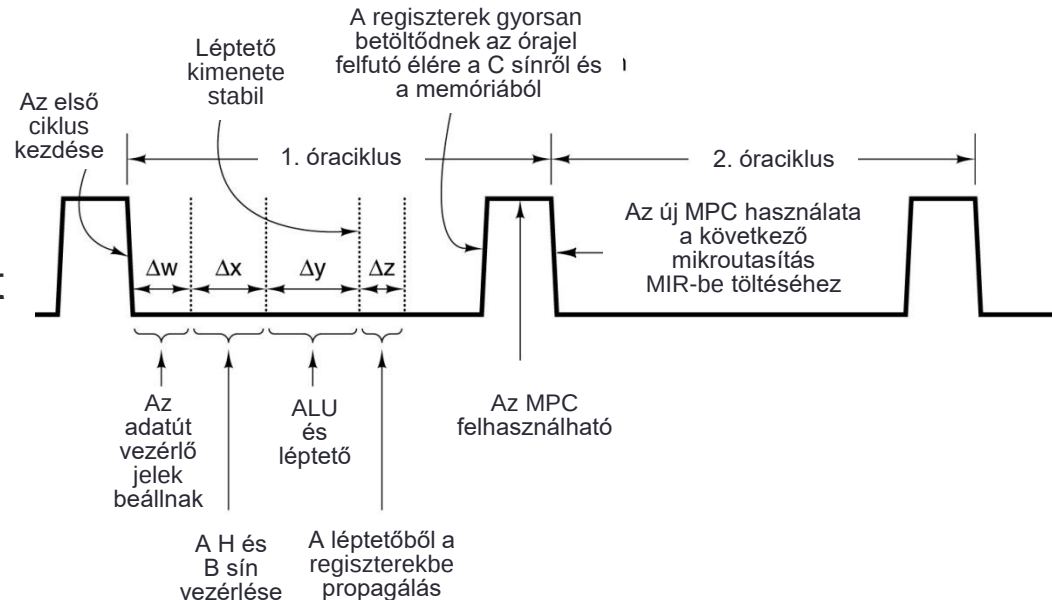
# Adatút

- Az olvasás és írás egy cikluson belül különböző időpillanatban hajtódnak végre
  - Amikor kiválasztódik, hogy melyik legyen az ALU jobb oldali bemenete
    - B sínre kerül az értéke
    - Ott marad majdnem a ciklus végéig
  - Az ALU elvégzi a feladatát
    - Az eredmény a léptetőn keresztül a C sínre küldi
  - Amikor az ALU és a léptető kimenete stabil
    - Egy órajel hatására a C sín tartalma egy vagy több regiszterben eltárolódik
- Az adatút pontos időzítése lehetővé teszi egy regiszter egy cikluson belüli olvasását és írását

# Adatút időzítése

Forrás: Tanenbaum, Structured Computer Organization, Fifth Edition, (c) 2006 Pearson Education, Inc. All rights reserved. 0-13-148521-0

- Minden óracikluskor egy rövid impulzus keletkezik
  - Származhat a fő órától
- A lefutó élen beállnak a kapukat irányító bitek
  - $\Delta w$  idő alatt bekövetkezik
- Kiválasztódik a regiszter és a B sínre kapcsolódik
  - $\Delta x$  idő múlva lesz az értéke stabil
- Az ALU és a léptető végrehajtja a műveletet az érvényes adatokon
  - $\Delta y$  idő múlva a kimenetek stabilak
- $\Delta z$  idő után az eredmények tovább terjednek a C sín mentén a regiszterekhez
- A felfutó élnél a regiszterek feltöltődnek
  - C sínről
  - Memóriából



**MPC és MIR később**

# Adatút időzítése

- A betöltésnek élvezéreltnak és gyorsnak kell lennie
- Az impulzus felfutó élénél
  - A regiszter és a B sín kapcsolata megszakad
  - Felkészül a következő ciklusra
- Meghatározott terjedési időt figyelembe kell venni
- A tervezés megköveteli
  - Szigorú időzítést
  - Hosszú óraciklust
  - Az ALU-n keresztüli minimális terjedési idő ismeretét
  - C sínról a regiszterbe való gyors betöltést
- Helyes működés!!!
  - Tervezés

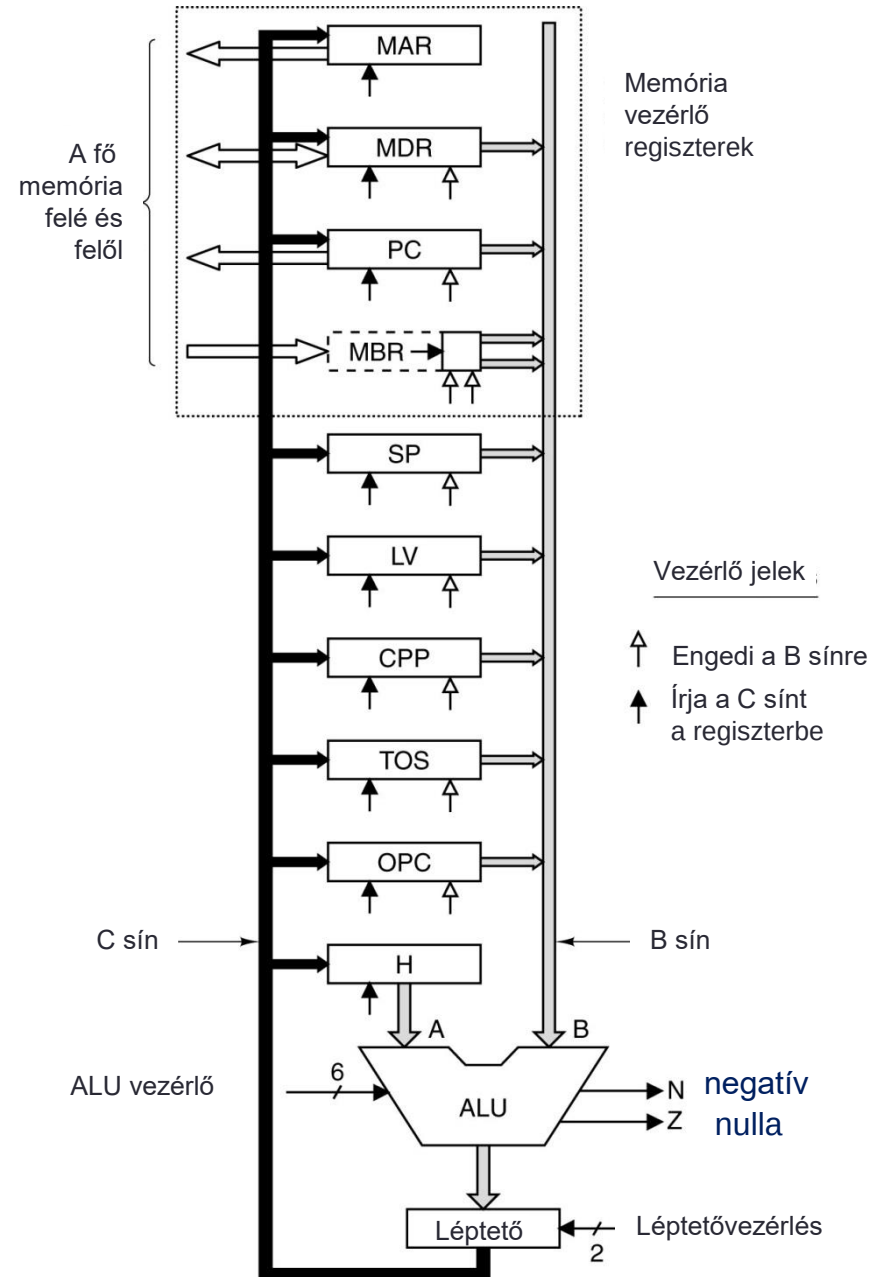


# Adatút időzítése

- Adatút-ciklus másik vizsgálata
  - Rész ciklusokra való darabolás
    - A vezérlőjelek beállnak ( $\Delta w$ )
    - A regiszterek a B sínre töltődnek ( $\Delta x$ )
    - Az ALU és a léptető működik ( $\Delta y$ )
    - Az eredmények a C sín mentén visszakerülnek a regiszterekhez ( $\Delta z$ )
  - A következő óraciklus felfutó élénél az eredmények eltárolódnak a regiszterekben
- A legjobb az, ha a részciklusok értelemszerűek
  - Nincs óra- vagy egyéb meghatározott jel, ami megmondja, hogy mikor mi működjön
  - Az ALU és a léptető mindvégig működik
    - A bemenetek bizonytalanok  $\Delta w + \Delta x$  ideig
    - A kimenetek bizonytalanok  $\Delta w + \Delta x + \Delta y$  ideig
  - Az egyedüli meghatározott jel az órajel lefutó és felfutó éle
    - Az adatút ciklusát indítja el illetve feltölti a regisztereket a C sínről

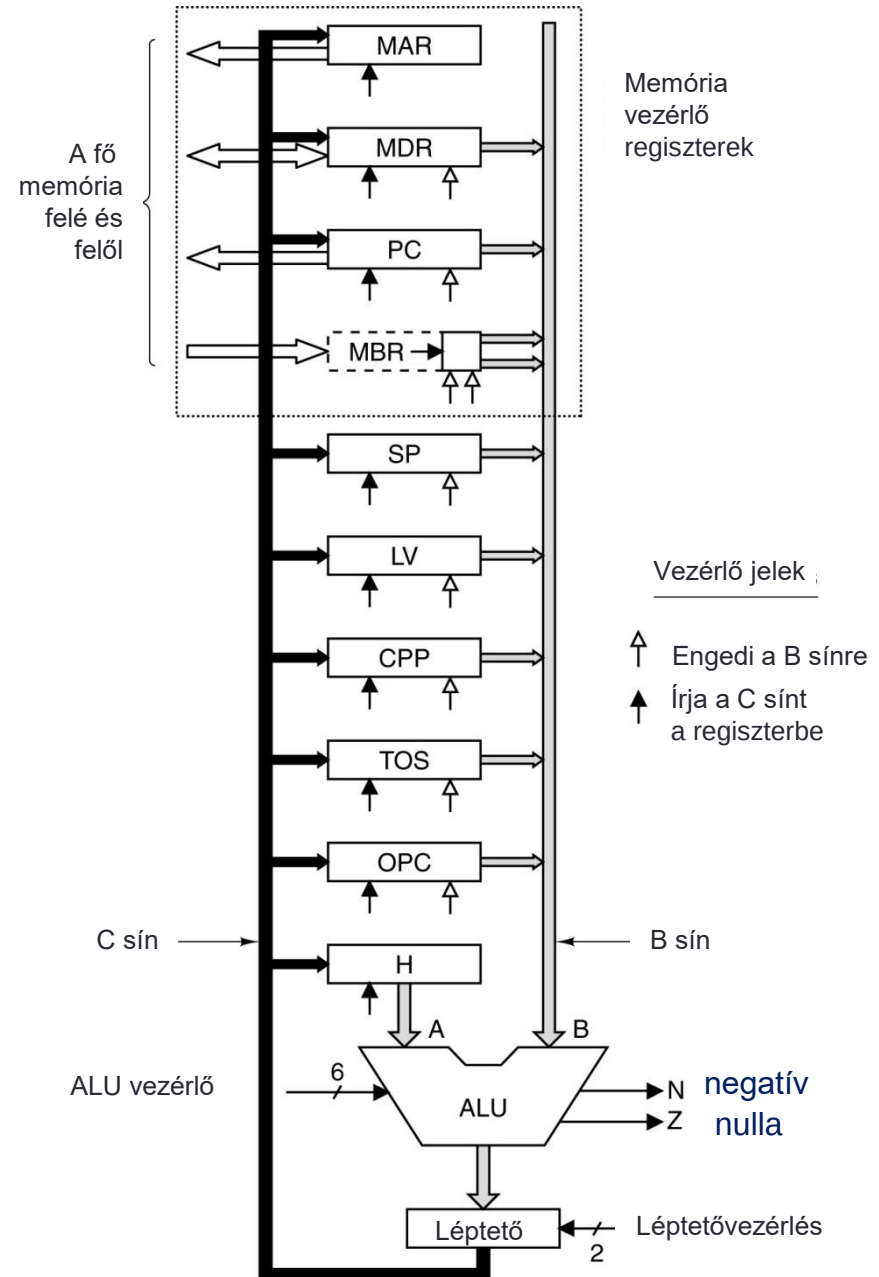
# A memóriaművelet

- Memóriával való kommunikáció
  - 32 bites szócímezésű memóriaport
    - Memory Address Register (memóriacím-regiszter) vezérli
    - Memory Data Register (memóriaadat-regiszter)
    - Olvasás/írás
  - 8 bites bájt címzésű memóriaport
    - PC vezérli
    - Memory Buffer Register (memóriapuffer-regiszter)
    - Csak olvasni tud



# A memóriaművelet

- A regisztereket egy vagy két vezérlőjel irányítja
  - Üres nyíl: a regiszter tartalmát engedi a B sínre
  - Teli nyíl: C sínről írja a regisztert
- Memóriaolvasás vagy –írás
  - Megfelelő regiszterek feltöltése
  - Olvasó- vagy írójel küldése a memória felé
    - Nincs az ábrán feltüntetve

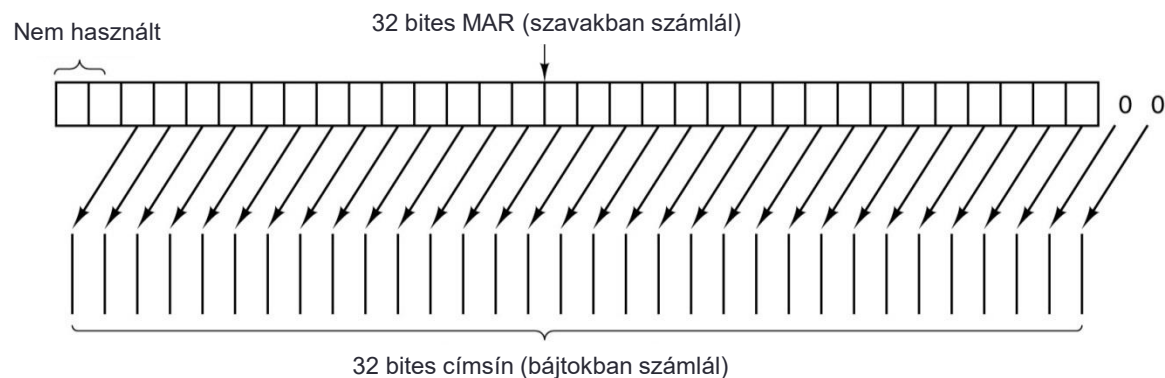


# A memóriaművelet

- MAR
  - Szócímekeket tartalmaz
  - Egymást követő szavakra hivatkozik
  - A kiolvasáskor az eredményt az MDR-be rakja
- PC
  - Bájt címekeket tartalmaz
  - Egymást követő bájtokra hivatkozik
  - MBR alsó 8 bitjére teszi
- A  $MAR \Leftrightarrow PC$ 
  - A memória két különböző részére való hivatkozás
  - Program Counter
- MAR/MDR
  - ISA-szintű adatszavak olvasása és írása
- PC/MBR
  - ISA-szintű bájt folyamból álló végrehajtható program olvasása
  - A többi regiszter szócímekeket használ

# A memóriaművelet

- Csak egy fajta fizikai memória van
  - Bájt-szervezésű
- MAR szavakban (4 bájt) számlál
  - Amikor a MAR a címsínre kerül
    - Nem képződik le a 32 címvonalra közvetlenül
    - 0. bit – címsín 2. vonala
    - 1. bit – címsín 3. vonala
    - ...
    - A legfelső 2 bitet eldobjuk
      - $2^{32}$  feletti címek

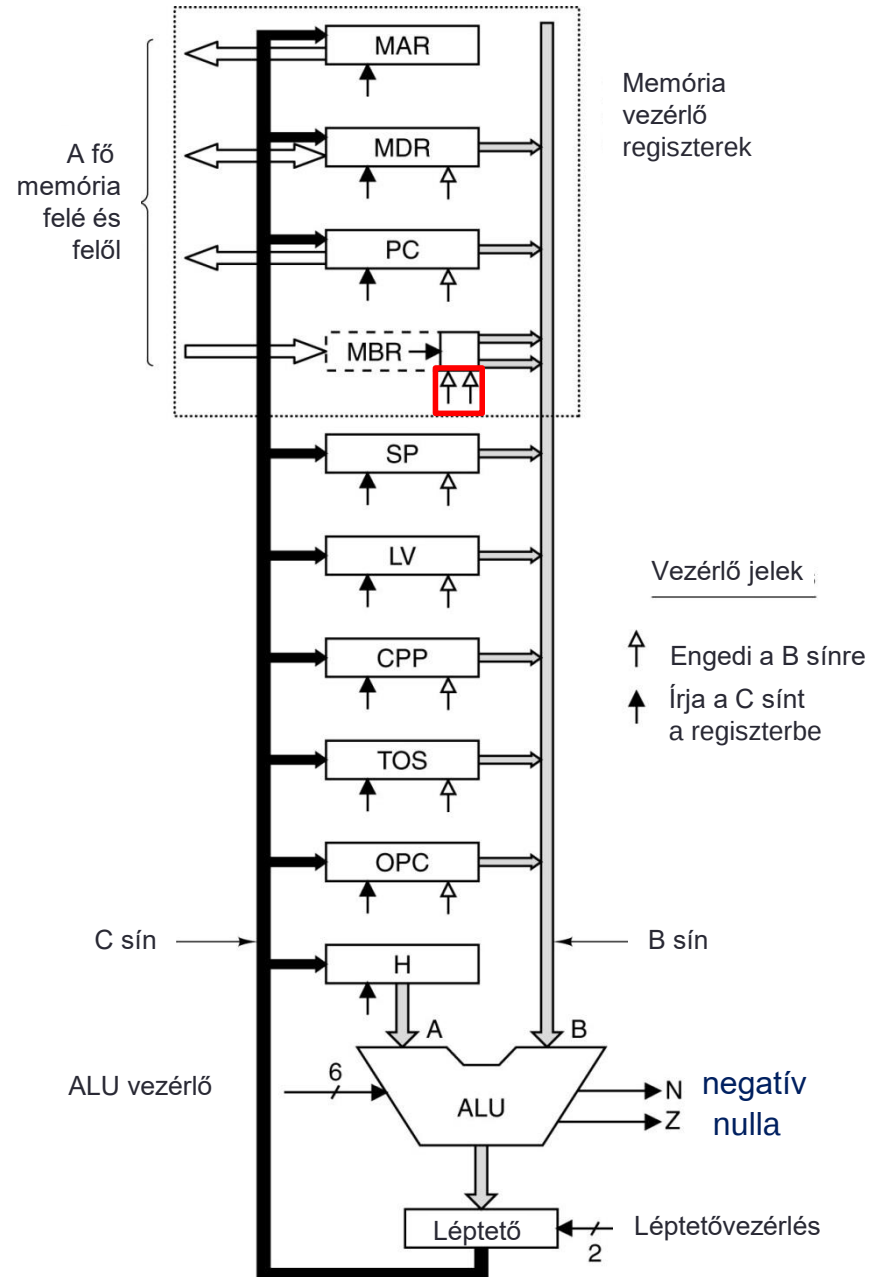


Forrás: Tanenbaum, Structured Computer Organization, Fifth Edition, (c) 2006 Pearson Education, Inc. All rights reserved. 0-13-148521-0

MAR = 1, akkor a sínre 4 kerül  
MAR = 2, akkor a sínre 8 kerül

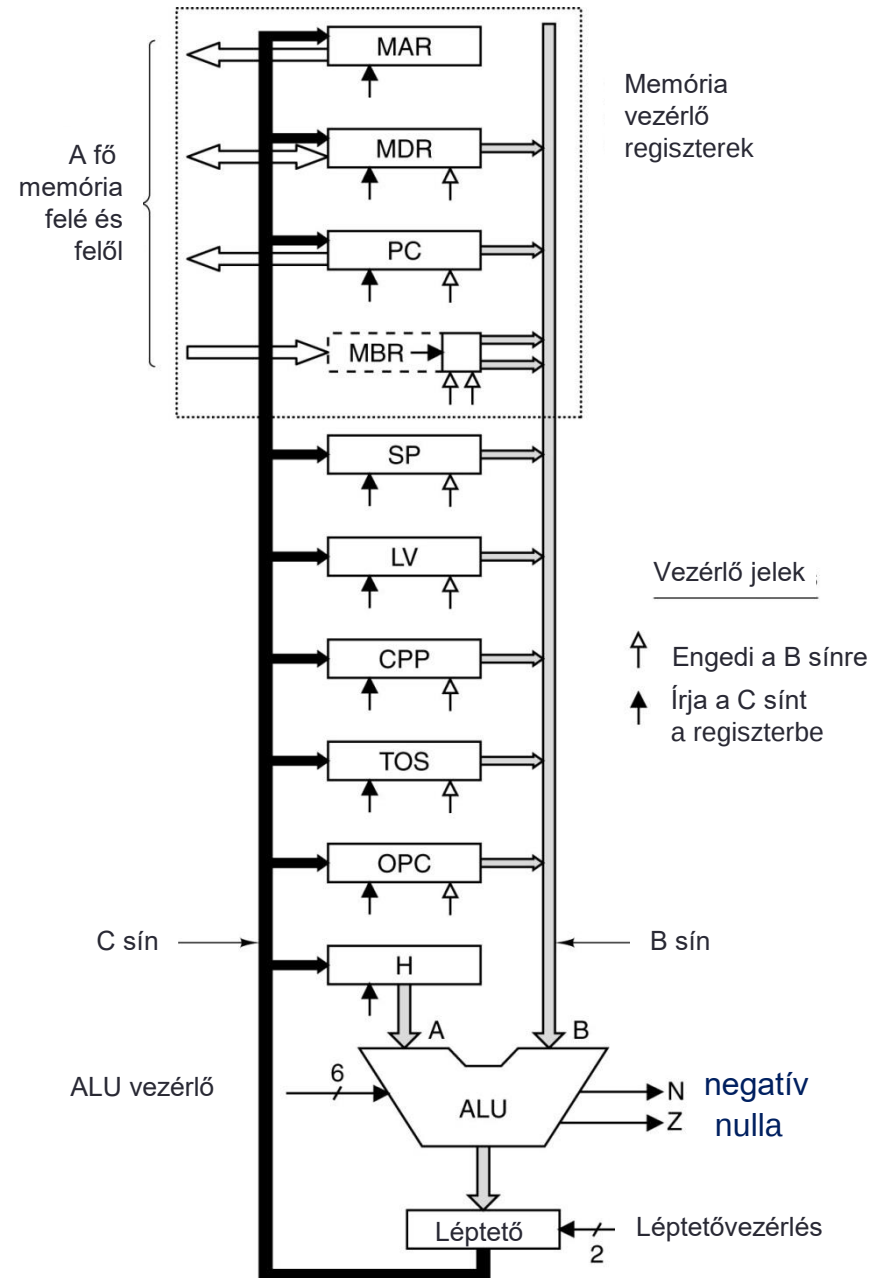
# A memória művelet

- 8 bites memóriaponton keresztüli olvasás
  - MBR
    - Kapuzható (másolható) a B sínre
      - Előjel nélkül
        - Alsó 8 bit és többi 24 biten 0-k
        - Táblázatok indexelése
      - Előjelesen
        - 32 bites szóvá való konvertálás
          - -128 és 127
          - Előjel bit másolása a felső 24 bitre
            - Előjel kiterjesztés
      - Két vezérlőjel



# Mikroutasítások

- Az adatút vezérléséhez 29 jelre van szükség
  - 9 jel vezérli az adatírást a C sínről a regiszterekbe
  - 9 jel vezérli a regiszterek engedélyezését a B sínre az ALU bemenet számára
  - 8 jel vezérli az ALU és a léptető tevékenységeit
  - 2 jel (nem látható) mutatja a memóriaolvasását/írást MAR/MDR-n keresztül
  - 1 jel (nem látható) mutatja a memóriabetöltést PC/MBR-n keresztül



# Mikroutasítások

- A 29 vezérlőjel értéke határozza meg az adatút egy ciklusának a műveleteit
  - Regiszterek értékének a B sínre való másolása
    - Csak egy regiszter
  - Jelek ALU-n és a léptetőn keresztüli továbbítása
  - Az eredmény C sínre való vitele
  - A C sínen lévő eredmény írása
    - megfelelő regiszterbe vagy regiszterekbe
  - Ezután, ha a memóriaolvasás jel be van állítva
    - A memóriaművelet az adatút ciklus végén kezdődik
      - MAR vagy PC feltöltődnek
    - Az adat a következő ciklus legvégén lesz MBR-ben vagy MDR-ben
      - Csak a rákövetkező ciklusban használható fel!!
      - Mindaddig, amíg az új értékek nem töltődnek be, addig a régi értékek használhatóak



# Mikroutasítások

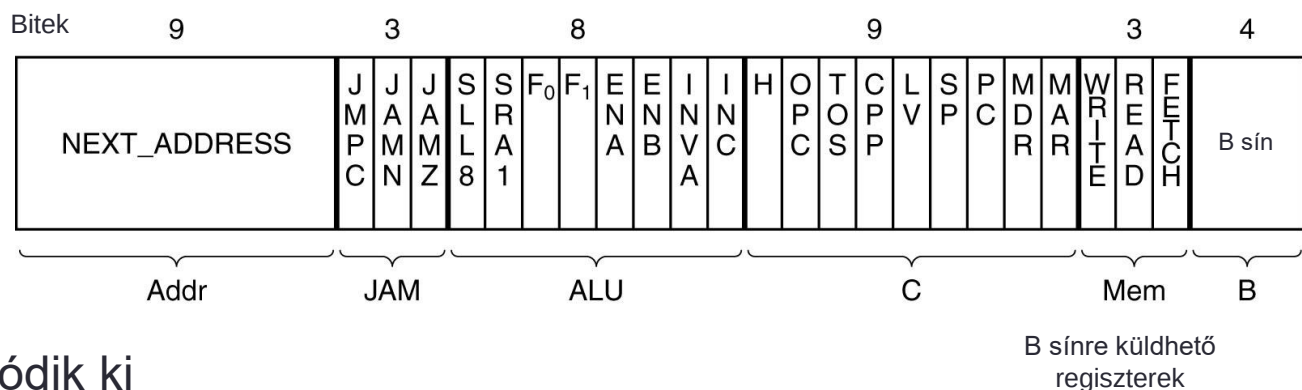
- Bitek számának a csökkentése
  - Lehetséges források kiválasztása és B sínre való küldése
  - Kilenc lehetséges regiszter van
    - 4 biten megoldható a B sín vezérlése
    - Dekóderrel előállíthatjuk a 16 vezérlőjelet
      - 7-et nem használunk ( $16 - 9 = 7$ )
    - Egy regiszter „felszabadításával” 3 bit elegendő lenne a feladat végrehajtásához
- 24 bittel vezéreljük az adatutatót
  - Csak egy ciklusra vezérli
  - Vezérlés második része
    - Mit akarunk csinálni a következő ciklusban?

# Mikroutasítások

- Formátum létrehozása a végrehajtandó műveletek leírására

- 24 vezérlő bit
- NEXT\_ADDRESS
  - Lehetséges következő mikroutasítás címe

- JAM
  - A következő mikroutasítás hogyan választódik ki



- 36 jel – 36 bites mikroutasítás

- Sorrend

0 = MDR    5 = LV  
 1 = PC    6 = CPP  
 2 = MBR    7 = TOS  
 3 = MBRU    8 = OPC  
 4 = SP    9-15 Nincs

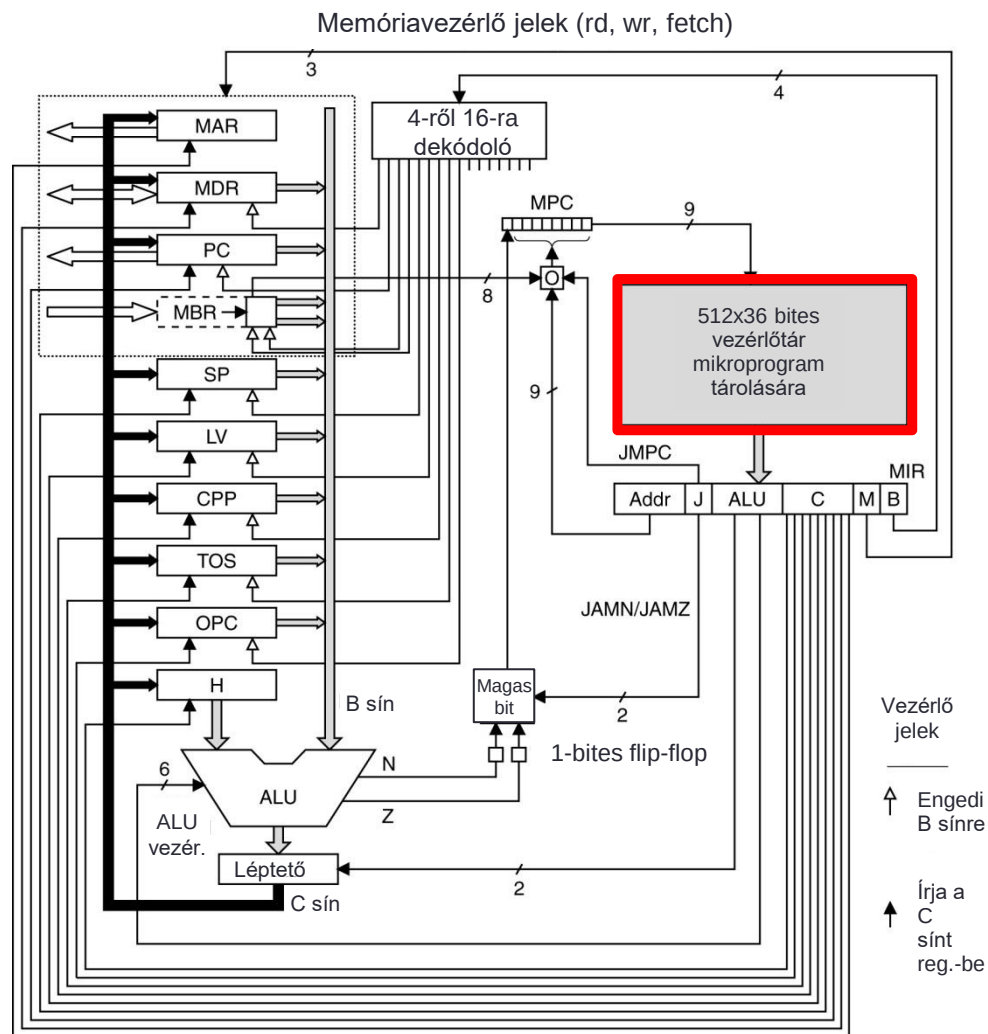


# Mikroutasítás- vezérlés: Mic-1

## • Adatút és vezérlőrés

### • Vezérlőtár

- Megvalósítása
  - Mikroprogramot tartalmazó memória
  - Nem főtár
- Logikai kapuk halmaza
- Memória, amely mikroutasításokat tartalmaz
- 512 szót tartalmaz
  - Minden szó 36 bites mikroutasításból áll



# Mikroutasítás- vezérlés: Mic-1

- A főmemóriában lévő utasítások végrehajtása címsorrendben történik
  - Kivéve az elágazásokat
- A mikroutasítások végrehajtása nem címsorrendben történik
  - Minden mikroutasítás előírja, hogy mi fogja követni
- A vezérlőtár csak olvasható memória
  - Nincs szüksége olvasó- és írójelekre
- Memóriacím-regiszter
  - MPC (MicroProgram Counter?)
    - Mikroutasítás számláló???
- Memóriadat-regiszter
  - MIR (MicroInstruction Register)
    - Tartalmazza az érvényes mikroutasítást
    - Bitjei meghajtják az adatutat működtető vezérlőjeleket



# Mikroutasítás- vezérlés: Mic-1

## Működés

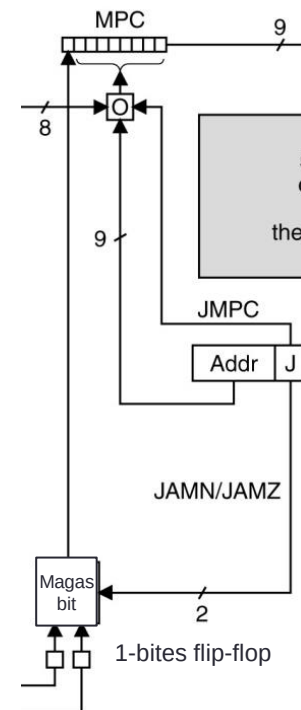
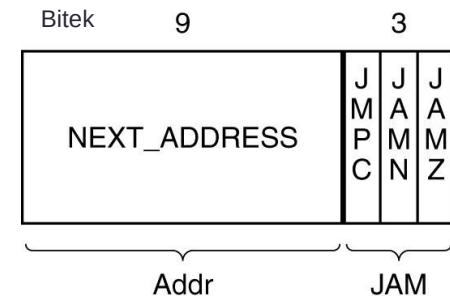
1. Az óraciklus kezdetén
  - A MIR feltöltődik az MPC által mutatott vezérlőtárbeli szóval
  - $\Delta w$  idő alatt
2. A mikroutasítás MIR-be kerülése után
  - Különböző jelek kerülnek az adatútra
  - Egy regiszter kikerül a B sínre
  - Az ALU megtudja, hogy milyen műveletet kell végrehajtani
  - $\Delta w + \Delta x$  idő után az ALU bemenetei stabilak lesznek
3. Az ALU, N, Z és a léptető kimenete stabil
  - N és Z értéke két 1-bites flip-flopban tárolódnak el
  - Az ALU kimenete a léptetőbe töltődik
  - $\Delta w + \Delta x + \Delta y$
4. A léptető kimenete a C sínen keresztül eléri a regisztereket
  - A regiszterek és az N és Z flip-flopok betöltéséből áll
  - Az óra emelkedő éle után befejeződik
    - Minden eredmény el van mentve
    - Előző memóriaművelet eredményei rendelkezésre állnak
    - MPC fel van töltve

# Mikroutasítás- vezérlés: Mic-1

## Működés

Forrás: Tanenbaum, Structured Computer Organization, Fifth Edition, (c) 2006 Pearson Education, Inc. All rights reserved. 0-13-148521-0

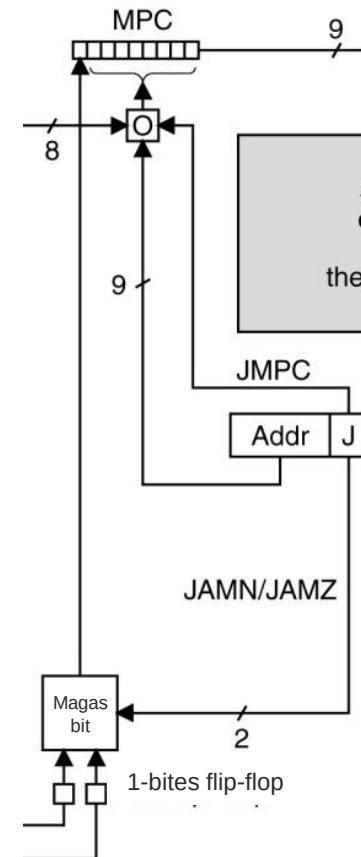
- Párhuzamosan meg kell határozni, hogy melyik mikroutasítást kell végrehajtani ezután
- A MIR feltöltődése és stabilizálódása után
  - A 9 bites NEXT\_ADDRESS az MPC-be másolódik
  - Közben megvizsgálja a JAM értékét
    - 000 esetén nem kell csinálni semmit
    - JAMN = 1 esetén
      - Az N flip-flop OR kapcsolattal kerül az MPC legmagasabb bitjére
    - JAMZ = 1 esetén
      - Az Z flip-flop OR kapcsolattal kerül az MPC legmagasabb bitjére
    - JAMN = JAMZ = 1
      - Mindkettő OR kapcsolattal az MPC legmagasabb bitjére kerül
    - Az N és Z flip-flopra szükség van
      - Emelkedő órajelnél a B sín már nincs vezérelve
      - Az ALU kimenetei nem biztos, hogy helyesek



# Mikroutasítás- vezérlés: Mic-1

## Működés

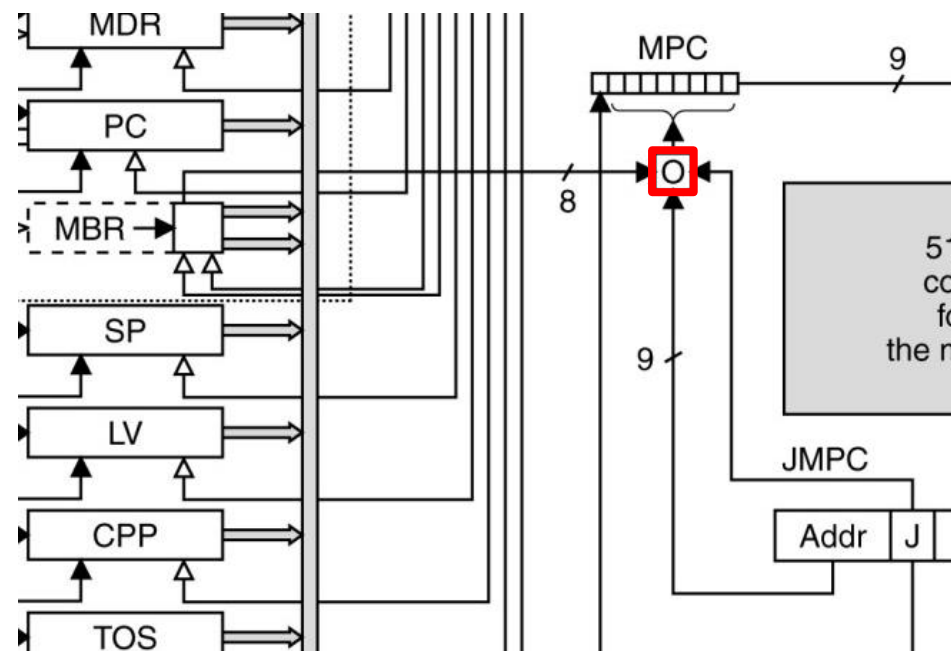
- A magas bit kiszámítása
  - $F = (JAMZ \text{ AND } Z) \text{ OR } (JAMN \text{ AND } N) \text{ OR } NEXT\_ADDRESS[8]$
- Amikor JAMN vagy JAMZ értéke 1, akkor a lehetséges címek
  - NEXT\_ADDRESS
  - NEXT\_ADDRESS OR 0x100
    - Ha  $NEXT\_ADDRESS \leq 0xFF$



# Mikroutasítás- vezérlés: Mic-1

## Működés

- JMPC = 1 esetén
  - MBR 8 bitjével bitenként OR műveletet hajt végre a NEXT\_ADDRESS alsó 8 bitjével
  - Az eredmény az MPC-be kerül
  - A NEXT\_ADDRESS értéke ilyenkor általában
    - 0x000 vagy 0x100
- JMPC = 0 esetén
  - Csak áteresztí NEXT\_ADDRESS értékét
- Több utas elágazás (ugrás) hatékony megvalósításánál fontos



Forrás: Tanenbaum, Structured Computer Organization, Fifth Edition, (c) 2006 Pearson Education, Inc. All rights reserved. 0-13-148521-0

# Mikroutasítás- vezérlés: Mic-1

## Működés

- Az MBR egy műveleti kódot tartalmaz tipikusan
  - A JMPC használatával egyértelműen ki tudjuk választani a következő mikroutasítást
    - Hasznos, amikor egy betöltött műveleti kódtól függő tevékenységet egy másiknak kell követnie
      - Gyors és közvetlen ugrást lehet végrehajtani

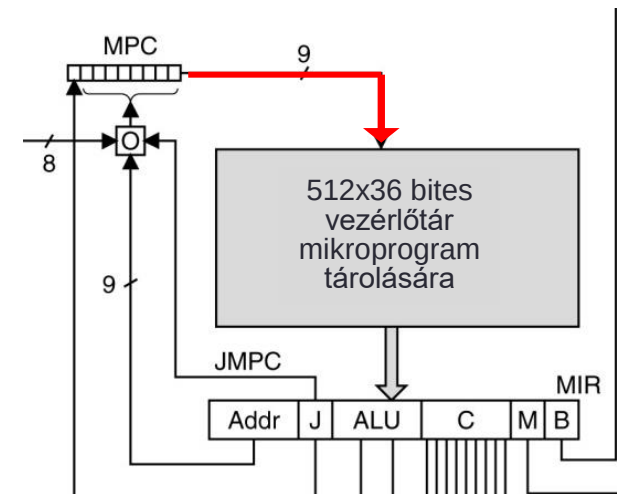
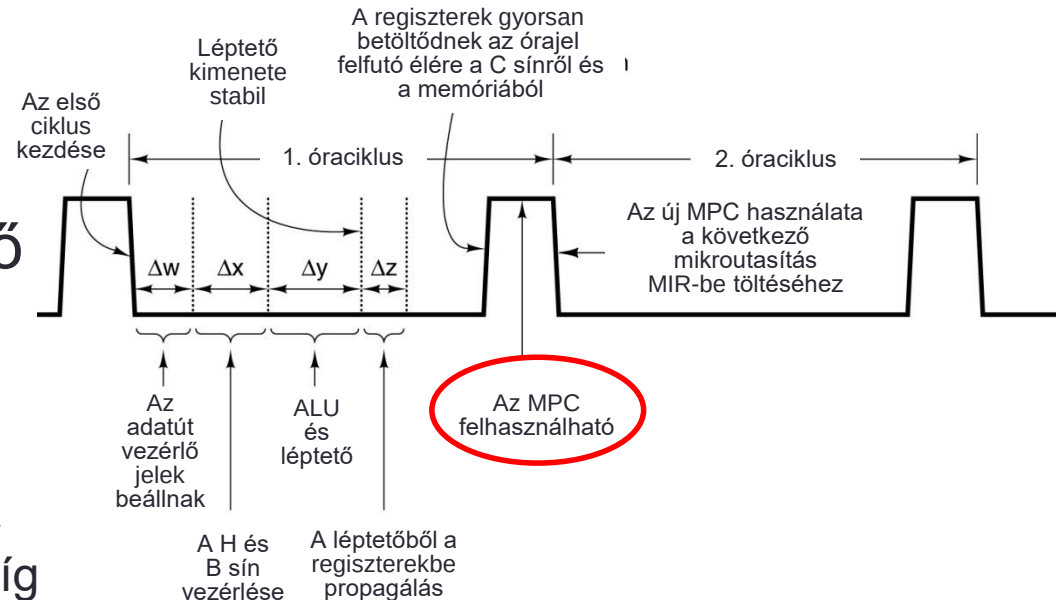


# Mikroutasítás- vezérlés: Mic-1

Forrás: Tanenbaum, Structured Computer Organization, Fifth Edition, (c) 2006 Pearson Education, Inc. All rights reserved. 0-13-148521-0

## Működés

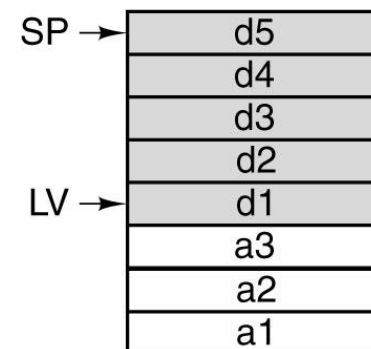
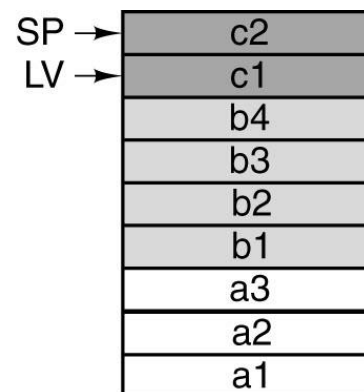
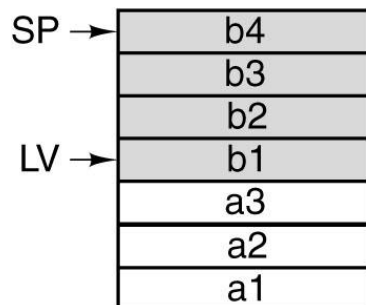
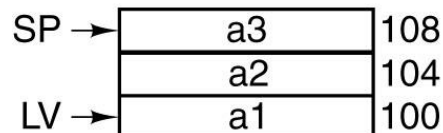
- Az időzítés fontos
  - [31-es fólia](#)
- Amikor az MBR elérhető
  - Az MPC betöltődik a következő mikroutasítás előkészítéséhez
  - MPC megkapja az értékét
    - Addig nem töltődik be, amíg azok a regiszterek nincsenek betöltve, amelyekről függ
      - MBR, N és Z
- Az órajel lefutásával az MPC megcímzi a vezérlőtárat
  - Új ciklus kezdődhet



# ISA-példa: az IJVM

- Vermek
  - Lokális változók tárolásának a problémája
    - Eljárások, metódusok
    - Elérhetőek az adott érvényességi körön belül
    - Megszűnik az elérhetőségük, amikor pl. kilépünk az eljárásból
  - Az eljárást önmagát is hívhatja (rekurzió)
    - Nem adhatunk abszolút memóriacímeket
  - LV (Local Variable) regiszter a lokális változók kiindulási pontjára mutat
    - Hivatkozás az eltolás mértékének megadásával LV-től
  - SP (Stack Pointer) adott eljárás lokális változói közül a legmagasabb szóra mutat
  - Lokális változó mező
    - LV és SP valamint az azok között lévő adatstruktúra

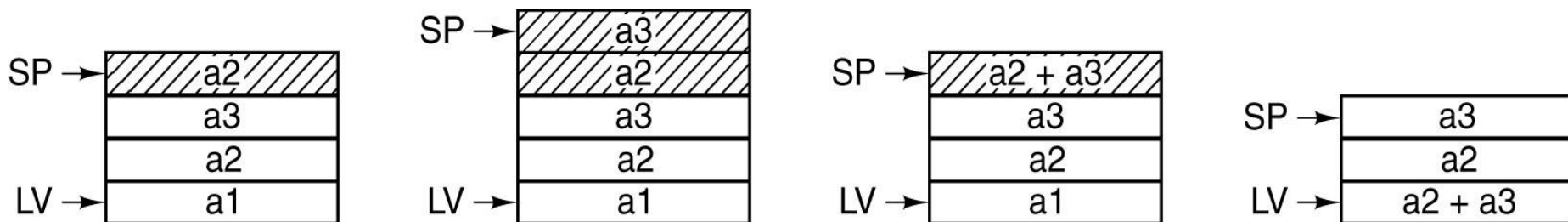
Forrás: Tanenbaum, Structured Computer Organization, Fifth Edition, (c) 2006 Pearson Education, Inc. All rights reserved. 0-13-148521-0



# Vermek

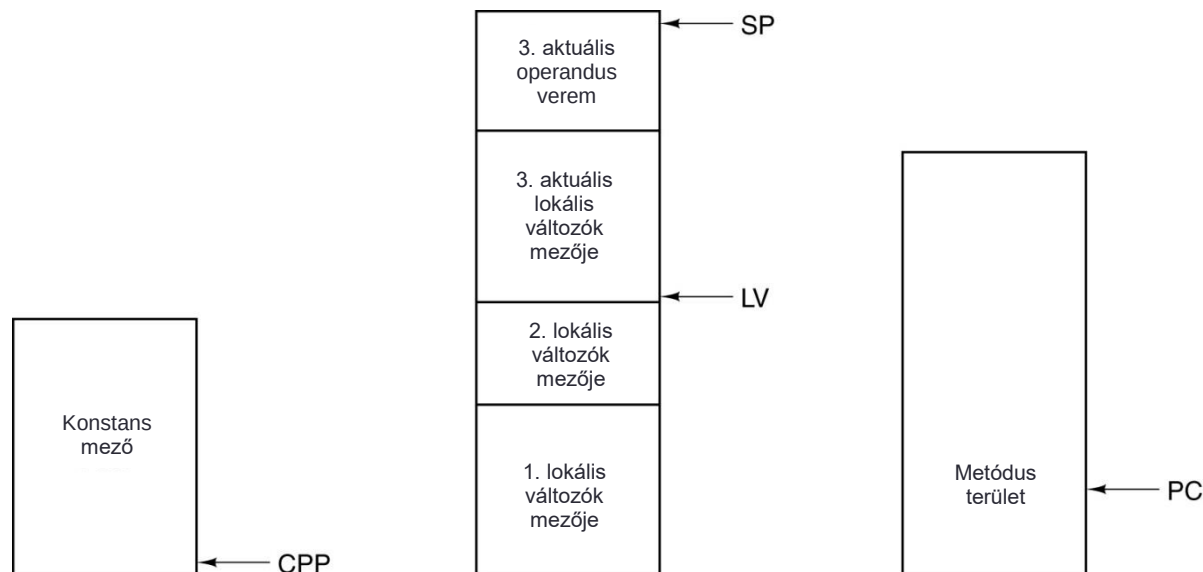
- Használhatók egy aritmetikai művelet operandusainak a tárolására is
  - Operandus verem

Pl.  $a1 = a2 + a3$



# Az JVM memóriamodellje

- A memória két módon látható
  - 4 294 967 296 bájtos tömb (4 GB)
  - 1 073 741 824 szavas tömb
- Nincsenek közvetlenül látható abszolút memóriacímek
- Indexeléssel érheti el a memóriát



Forrás: Tanenbaum, Structured Computer Organization, Fifth Edition, (c) 2006 Pearson Education, Inc. All rights reserved. 0-13-148521-0

# Az JVM memóriamodellje

## Konstans mező

- JVM program nem írhatja
- Betöltés a program memóriába kerülésével
- CPP (Constant Pool Pointer)
  - Első szavának a címe

## Lokális változók mezője

- Metódus hívásakor egy terület foglalódik le
- Elején az argumentumok
- Nincs benne az operandusverem
- LV
  - Első címhely tárolása

# Az JVM memóriamodellje

## Operandusverem

- A verem mező nem lehet túl nagy
- Lokális változók mezője fölött van lefoglal
- SP
  - A verem legfelső szavának a címe
  - Változik

## Metódus mező

- A programot tartalmazza
- „szöveg”
- PC (Program Counter)
  - Utasítás számláló
- Bájtömbként kezeljük

# IJVM utasításkészlete (részlet)

| Hex  | Mnemonic                       | Jelentés                                                               |
|------|--------------------------------|------------------------------------------------------------------------|
| 0x10 | <b>BIPUSH <i>byte</i></b>      | Betesz a <i>byte</i> -ot a verembe                                     |
| 0xA7 | <b>GOTO <i>offset</i></b>      | Feltétel nélküli ugrás <i>offset</i> -re                               |
| 0x60 | <b>IADD</b>                    | Kivesz a veremből két szót, az összegüket a verembe teszi              |
| 0x99 | <b>IFEQ <i>offset</i></b>      | Kivesz a veremből egy szót, ha 0, akkor <i>offset</i> -re ugrik        |
| 0x9F | <b>IF_ICMPEQ <i>offset</i></b> | Kivesz a veremből két szót, ha egyenlők, akkor <i>offset</i> -re ugrik |
| 0x15 | <b>ILOAD <i>varnum</i></b>     | Betesz <i>varnum</i> -ot a verembe                                     |
| 0x36 | <b>ISTORE <i>varnum</i></b>    | Kivesz a veremből egy szót, és eltárolja <i>varnum</i> -ba             |
| 0x64 | <b>ISUB</b>                    | Kivesz a veremből két szót, a különbségüket a verembe teszi            |
| 0x00 | <b>NOP</b>                     | Nem csinál semmit                                                      |
| 0x5F | <b>SWAP</b>                    | A verem két felső szavát megcseréli                                    |

# Java fordítása IJVM-re

Forrás: Tanenbaum, Structured Computer Organization, Fifth Edition, (c) 2006 Pearson Education, Inc. All rights reserved. 0-13-148521-0

|             |    |              |                |                |
|-------------|----|--------------|----------------|----------------|
| i = j + k;  | 1  | ILOAD j      | // i = j + k   | 0x15 0x02      |
| if (i == 3) | 2  | ILOAD k      |                | 0x15 0x03      |
| k = 0;      | 3  | IADD         |                | 0x60           |
| else        | 4  | ISTORE i     |                | 0x36 0x01      |
| j = j - 1;  | 5  | ILOAD i      | // if (i == 3) | 0x15 0x01      |
|             | 6  | BIPUSH 3     |                | 0x10 0x03      |
|             | 7  | IF_ICMPEQ L1 |                | 0x9F 0x00 0x0D |
|             | 8  | ILOAD j      | // j = j - 1   | 0x15 0x02      |
|             | 9  | BIPUSH 1     |                | 0x10 0x01      |
|             | 10 | ISUB         |                | 0x64           |
|             | 11 | ISTORE j     |                | 0x36 0x02      |
|             | 12 | GOTO L2      |                | 0xA7 0x00 0x07 |
|             | 13 | L1: BIPUSH 0 | // k = 0       | 0x10 0x00      |
|             | 14 | ISTORE k     |                | 0x36 0x03      |
|             | 15 | L2:          |                |                |

# Példa a megvalósításra

- Hogy néz ki az a program, ami a mikroarchitektúrán fut és értelmezi a makroarchitektúrát?
- Hogyan működik?
- Mikroutasítások és jelölésrendszer
  - A vezérlőtárat le lehetne írni binárisan
  - Szükség van egy jelölésrendszerre
    - Pl. címek kiválasztása
  - Egyetlen sor
    - Tevékenység meghatározása, ami egy óraciklusban történik
    - Ciklusról-ciklusra való vezérlés
    - Hack-elés
      - Sok dolog rejtve marad
      - Gyorsítás

# Mikroutasítások és jelölésrendszer

- Művelet
  - SP értékét növelni akarjuk
  - egy olvasást kezdeményezünk
  - A következő utasítás a 122-es helyen lévő vezérlő tárbeli utasítás legyen
  - ReadRegister=SP,ALU=INC,WSP, Read, NEXT\_ADDRESS=122
    - WSP: írjunk az SP regiszterbe
  - Összhatás
    - $SP = SP + 1$ ; rd
- Micro Assembly Language (MAL)
  - Minden ciklusban egy regiszter írható
  - $MDR = SP$ 
    - SP-t bemásolja MDR-be

# Mikroutasítások és jelölésrendszer

- $MDR = H + SP$ 
  - $MDR = SP + H$
- Megengedett műveletek
  - Ki lehet egészíteni
    - $\ll 8$
    - Egy bájjal való balra léptetés
- SOURCE
  - MDR, PC, MBR, MBRU, SP, LV, CPP, TOS, OPC
  - B sínen
- DEST
  - MAR, MDR, PC, SP, LV, CPP, TOS, OPC, H
  - C sínen

|                            |
|----------------------------|
| DEST = H                   |
| DEST = SOURCE              |
| DEST = $\overline{H}$      |
| DEST = $\overline{SOURCE}$ |
| DEST = H + SOURCE          |
| DEST = H + SOURCE + 1      |
| DEST = H + 1               |
| DEST = SOURCE + 1          |
| DEST = SOURCE - H          |
| DEST = SOURCE - 1          |
| DEST = -H                  |
| DEST = H AND SOURCE        |
| DEST = H OR SOURCE         |
| DEST = 0                   |
| DEST = 1                   |
| DEST = -1                  |

# Mikroutasítások és jelölésrendszer

- Egy ciklus alatt nem végrehajtható ezért nem megengedett
  - $PI. MDR = SP + MDR$  vagy  $H = H - MDR$
- Többszörös tárolás
  - $SP = MDR = SP + 1$
- Memória írás olvasás
  - rd, rw: MAR/MDR
  - fetch: PC/MBR
- Példa, hogyan ne csináljuk
  - $MAR = SP; rd$ 
    - Átad egy értéket a memóriából az MDR-be a második utasítás végén
  - $MDR = H$ 
    - Betesz egy értéket az MDR-be
  - Az eredmény definiálatlan lesz

# Mikroutasítások és jelölésrendszer

- Címkére ugrás
  - Következő cím meghatározása
    - Egymás után írt sorok, egymás után lesznek végrehajtva
  - goto label
- Elágazás
  - Valami egyenlő-e 0-val?
    - TOS = TOS
    - Z bitje az ALU-nak
    - if (Z) goto L1; else goto L2
    - Z = TOS; if (Z) goto L1; else goto L2
- JMPC bit használata
  - goto (MBR OR value)

# IJVM megvalósítása Mic-1 felhasználásával (részlet)

Forrás: Tanenbaum, Structured Computer Organization, Fifth Edition, (c) 2006 Pearson Education, Inc. All rights reserved. 0-13-148521-0

| Címke | Műveletek                        | Megjegyzések                                                 |
|-------|----------------------------------|--------------------------------------------------------------|
| Main1 | PC=PC+1;fetch;<br>goto (MBR)     | MBR-ben a műveleti kód;<br>veszi a következő bájtot,elágazás |
| nop1  | goto Main1                       | Semmit nem csinál                                            |
| iadd1 | MAR=SP=SP-1;rd                   | Beolvassa a verem teteje alatti szót                         |
| iadd2 | H=TOS                            | H=verem teteje                                               |
| iadd3 | MDR=TOS=MDR+H;<br>wr; goto Main1 | Összeadja a két felső szót;<br>beírja a verem tetejére       |
| isub1 | MAR=SP=SP-1;rd                   | Beolvassa a verem teteje alatti szót                         |
| ...   | ...                              | ...                                                          |

# IJVM megvalósítása Mic-1 felhasználásával

- Hogyan dolgozik az értelmező
  - MBR = 0x60

|      |             |                                                           |
|------|-------------|-----------------------------------------------------------|
| 0x60 | <b>IADD</b> | Kivesz a veremből két szót, az összegüket a verembe teszi |
|------|-------------|-----------------------------------------------------------|

- Fő ciklus
  - PC = PC + 1; fetch; goto (MBR)
  - Növeljük a PC-t
    - A műveleti kódot követő első bájt címét fogja tartalmazni
    - Következő bájt betöltése MBR-be
    - MBR-ben lévő címkére ugrunk
  - MAR = SP = SP – 1; rd
    - Verem teteje alatti szót beolvassa
  - H = TOS
    - H tartalmazz a verem tetejét
  - MDR = TOS = MDR + H; wr goto Main1
  - iadd1, iadd2, iadd3
    - MPC megkapja az előző utasítás NEXT\_ADDRESS mezőjéből

